

LumoTree: Path-Parallel Speculative Verification for Hybrid Language Models

Zhiyuan Ma

zhiyuanma0520@gmail.com

Abstract—Tree speculative decoding for hybrid language models must preserve one coherent continuation across recurrent, convolution, and attention state. We present LumoTree, a verifier that executes recurrent paths in parallel, reuses state tiles within each path, and coordinates native recurrent replay, convolution-history gathering, and attention-cache remapping through a shared logical tree. The implementation combines fused candidate selection, GPU-resident acceptance, and grouped split-K attention. Component experiments establish candidate-selection parity and recurrent agreement within specified paired error bounds. Full-model correctness remains unestablished: across independent processes, the native reference preserves greedy decisions on all 84 tested paths but reproduces joint target and ordered draft selections on only 68. Exploratory Qwen3.8-27B NVFP4 deployments on GB10 attain 27.27–29.09 pooled tokens/s across three LumoTree builds, compared with 26.89 for SGLang EAGLE, on two shared SWE-bench Verified Astropy tasks with nonempty patches. These retrospective observations characterize the recorded deployments; they do not establish a same-stack speed advantage or general task-quality preservation.

Index Terms—speculative decoding, Gated DeltaNet, state commitment, numerical reproducibility, language-model serving

I. INTRODUCTION

Speculative decoding reduces serial target-model invocations by verifying draft tokens in parallel. Its sampling guarantee assumes that verification supplies the intended target probabilities. Tree speculation expands the candidate set, but it does not relax that assumption [1]–[3]. For recurrent-hybrid models, realizing the verifier requires more than an attention mask: a candidate must inherit the state of its ancestors, and the next serving iteration must inherit exactly the path selected for continuation [4], [5].

LumoTree executes recurrent paths in parallel, reuses state tiles across updates within each path, and coordinates the accepted continuation across recurrent, convolution, and attention state. An ancestry-aware attention path and a device acceptance walk use the same logical tree as recurrent verification and state publication. A shared descriptor carries ancestry and logical-to-physical mappings through each stage [6], [7]. This system boundary matters because a verifier can compute plausible candidate logits yet publish the wrong state for the next iteration.

The central execution choice is to verify branches in temporary state and replay only the selected path into the native running state. The verifier scans branch paths, while the fixed-shape committer replays accepted operands through native GDN updates in a captured graph. Their finite-precision agreement is a separate requirement, not assumed from the recurrence.

This choice exposes concrete optimization questions: how much branch state to cache, when to recompute ancestry, how to organize independent GPU work, and how to keep full-vocabulary probability arithmetic on the device. The attention layout and accepted-state mappings constrain those changes as much as the recurrent equation does.

The paper studies three coupled design questions in recurrent-hybrid tree verification. *First*, how can branch-local computation produce one coherent continuation across recurrent, convolution, and attention state? We define a shared logical tree and coordinate state publication through its logical-to-physical mapping. *Second*, how should verification trade temporary state storage against repeated computation? We use path-parallel scans with shared boundary states and replay the accepted path into persistent state, building on established chain-scheduling and replay mechanisms [8], [9]. *Third*, how can this organization reduce serving overhead? We combine fused candidate selection, device-resident acceptance, and grouped split-K attention with a spine-contiguous KV layout. Component experiments and exploratory coding-agent deployments characterize the implementation. Cross-process differences in the native reference prevent establishing full-model continuation correctness; the workload observations do not establish a same-stack performance advantage.

The target workload is a long-running coding agent: generation alternates with repository inspection, tool execution, and tests, and a task can require many model requests. Performance therefore depends on useful completed work as well as token production. Our workload evaluation uses explicitly identified Astropy tasks from SWE-bench Verified [10]. We report the model, serving route, task cohort, and measurement definition together; a decode-rate change is not a task-completion speedup. Candidate-selection and attention tests complement the agent-workload measurements.

Tree verification and compact accepted-state reconstruction have direct prior art. Section II positions our implementation against those mechanisms. The architecture and kernel design are developed in Sections III–IV, and Section V separates optimization mechanisms from their measured scope. Measurement provenance is documented in Appendix A.

II. BACKGROUND AND RELATED WORK

A. Drafting, verification, and serving

Speculative sampling constructs an exact target sample from proposal and residual distributions, conditional on the probabilities actually supplied [1], [2]. SpecInfer supplies



Fig. 1. Tree verification and accepted-path commitment. (a) Drafting provides a candidate tree and proposal probabilities q . (b) Branch-local computation produces target probabilities p and retains temporary state. (c) Sampling selects a materialized path (here a, d); recurrent replay, convolution gathering, and KV remapping publish $S(mad)$. The newly sampled token b^* remains pending. One descriptor keeps logical ancestry and physical storage aligned.

the tree-verification framing; Sequoia, EAGLE-2, and OPT-Tree investigate tree construction and its performance consequences [3], [11]–[13]. EAGLE-3 and DFlash further change the drafter, through feature-conditioned autoregression and block-parallel diffusion proposals respectively [14], [15]. These drafting mechanisms are distinct from the recurrent verifier and state-publication choices studied here.

PagedAttention and SGLang address the scheduling, memory, and execution substrate [16], [17]. Marconi addresses prefix-cache admission and eviction for hybrid models [18]. Hybrid serving additionally manages recurrent-state pools alongside KV storage, as documented by SGLang’s hybrid-model integration [19], [20]. Thus neither a fast isolated recurrent kernel nor low transient state memory alone determines user-visible latency. Flash-Decoding splits the KV sequence and combines partial attention results; FlashInfer develops customizable attention with head/query fusion and split-KV execution; DeFT specializes KV grouping and partitioning to trees [21]–[23]. FastTree groups queries with shared contexts to reuse KV tiles and partitions tree work to control execution cost [24]. These attention methods establish precedents for on-chip reuse and tree-aware partitioning. Our modified FlashAttention-2 kernel combines ancestry-aware decode, grouped split-K execution, and a physical layout coordinated with accepted-path publication [6], [25].

B. Recurrent and hybrid speculation

Gated DeltaNet augments a decayed recurrent state with a data-dependent delta update [4]. Parallel delta-rule formulations provide relevant algebraic background [26]. Snakes and Ladders introduces activation replay for SSM speculative decoding, and STree also uses replay to recover continuation state [5], [27]. Stateful speculation is established territory: STree studies speculative trees for hybrid state-space models, SpecMamba targets an FPGA implementation, and component-aware self-speculation studies how hybrid components affect the design [5], [28], [29]. These works motivate an explicit

state lineage; they do not justify treating every recurrent update or numerical implementation as interchangeable.

C. Direct GDN comparisons

Trees from Marginals (Weaver) combines a factorized drafter with an autoregressive adapter. Its GDN verifier uses an ancestor-masked triangular solve and replays the selected path through a short recurrence at commit time; padded verification shapes support CUDA graphs [9]. Thus read-only speculative verification followed by accepted-path replay is direct prior art, even when the verification arithmetic differs from ours.

SpecLA keeps value tiles of recurrent state on chip across updates within a chain and schedules independent ready chains in parallel. Boundary states connect dependent chains. This directly overlaps both the state reuse and path scheduling in LumoTree. SpecLA buffers accepted factors for a later fused update, whereas our publication stage replays recorded operands through native GDN updates into running rows. Its evaluation uses a pure GDN model; our serving boundary additionally coordinates convolution and softmax attention [8]. SpecLA’s token-replay baseline regenerates factors, so its cost cannot be assigned to our replay of cached operands.

Bole derives a tree-structured closed form and evaluates it with a value-tiled finite-polynomial solver. It reuses intermediate tiles on chip, stores compact update factors, and reconstructs the accepted state. Its SGLang integration combines batch-wide verification budgeting with captured GPU execution. Evaluation includes unquantized Qwen3.5-27B on GB10 and replay of recorded OpenHands agent sessions [30]. This is a direct hybrid-serving comparison; its workload replay differs from running the coding agent on tasks, as in our evaluation.

Concretely, Bole’s correction system is $(I + G)U = R$, where G contains strict-ancestor interactions and R is the source correction. Tree depth d gives $G^{d+1} = 0$, so $U = \sum_{m=0}^d (-G)^m R$. This finite identity changes the arithmetic schedule from sequential updates to matrix propagation [30]. Our path preserves sequential updates and replays accepted

TABLE I
RECURRENT VERIFICATION AND CONTINUATION POLICIES. THESE
MECHANISM COMPARISONS DO NOT RANK THE SYSTEMS BY
PERFORMANCE.

Method	Verification work	Continuation state
Weaver [9]	Ancestor-masked triangular solve	Short selected-path recurrence
SpecLA [8]	Resident state tiles; parallel chains	Buffered factors; later fused update
Bole [30]	Finite-Neumann correction solve	Compact factors; state reconstruction
TreeWY [32]	Triangular solve in fused kernel	Prior-path reconstruction with next verify
ReplaySSM [33]	Corrected inputs; cached transitions	Deferred state update
OneLA [34]	Shared-base projections across beams	Compact per-beam transition records
LumoTree	Resident state tiles; parallel paths	Native replay; coordinated convolution and KV publication

operands; Section V-B explains why the numerical comparison remains an empirical question.

TreeWY expresses GDN tree verification as a triangular system and reconstructs the accepted state from pseudo-values. The author implementation fuses reconstruction of the previous accepted state with the next verification, reusing a resident state tile. Its GDN tree kernel supports CUDA graph capture; the paper’s wider-tree fallback to piecewise execution concerns ancestry masks in the softmax-attention integration [31], [32]. Its B200 evaluation uses greedy verification and disabled prefix caching. A comparison must therefore include fused commit work and distinguish kernel support from whole-model execution policy.

ReplaySSM caches recurrent inputs and delays state materialization. Its GDN path uses corrected inputs from a triangular solve, with commitment represented through buffer progress and later state updates [33]. **OneLA** addresses a related state-sharing problem in recommendation beam search: compact transition records and ancestry indices allow projection computation without reconstructing a full state for each beam, and a fused kernel shares the prompt-derived state across beams [34]. Its beam-search output and commitment policy differ from speculative acceptance. Table I compares state lifetimes; published speed ratios from different models and workloads are not combined into a ranking.

D. Design focus

LumoTree connects path-parallel recurrent computation to one accepted continuation across the hybrid model. The shared descriptor links path boundaries, convolution ancestry, attention visibility, physical KV slots, and the next drafter input. Native replay preserves the serving system’s recurrent-state ownership while selected-path gathering and remapping publish the other state components. Spine-first KV placement controls the physical attention reduction layout within this design, building on established floating-point reduction and

batch-invariance concerns [35]. Evaluation separates component behavior from complete agent-workload performance.

III. TREE-VERIFIER ARCHITECTURE

A. State and the execution boundary

Let m be a materialized token prefix: the tokens already consumed by the target’s forward computation. Write its persistent state as

$$S(m) = (R(m), C(m), K(m)), \quad (1)$$

where R collects recurrent matrices, C collects convolution histories, and K collects attention KV entries. The logical emitted prefix can also include a newly sampled token b that has not yet been consumed. The continuation boundary is then $(S(m), b)$, not a claim that $S(mb)$ has already been computed. This distinction is essential when counting the correction or bonus token while checking cache state.

A tree descriptor contains candidate token IDs, parent indices, depth/position information, attention visibility, and maps between logical nodes and physical slots. For a candidate node i , each target layer must consume the state obtained from the candidate’s root-to-parent path. At publication, the selected materialized path, pending-token convention, and next-drafter input row must agree. Figure 2 illustrates the three state surfaces coordinated by the verifier [6], [7].

B. The recurrence and path locality

For one GDN head, let $R \in \mathbb{R}^{d_v \times d_k}$. Queries and keys are in $\mathbb{R}^{d_k}$ and values in $\mathbb{R}^{d_v}$. With decay α_i and write strength β_i , a node applies the standard update [4]

$$R'_i = \alpha_i R_{\pi(i)}, \quad \delta_i = \beta_i (v_i - R'_i k_i), \quad (2)$$

$$R_i = R'_i + \delta_i k_i^\top, \quad o_i = R_i q_i, \quad (3)$$

where $\pi(i)$ is its parent. A previous node in the packed buffer need not be $\pi(i)$. The causal convolution likewise requires path predecessors rather than adjacent packed rows. The attention visibility relation must select the same logical prefix and ancestry. These requirements follow from the model’s computation; a correct attention mask cannot repair a wrong recurrent parent.

The implementation verifies nodes with branch-local scratch and makes the selected continuation durable through replay and remapping. This is an implementation route, not the only way to satisfy the contract [6], [7]. In particular, rejecting a branch requires that its contents cannot influence future computation; it does not require zeroing every scratch byte if future reads are correctly restricted.

C. A descriptor shared by verification and commitment

Figure 1 shows how a shared descriptor connects drafting to continuation. Native multi-token prediction (MTP) supplies a principal chain and runner-up candidates; suffix prediction extends selected paths within a bounded candidate capacity [36], [37]. Combining retrieval and neural drafting has prior art, including SAM Decoding [38]. The verifier derives attention



Same selected materialized prefix on all three surfaces; correction token b may remain pending for the next forward.

Fig. 2. The accepted-prefix interface. Recurrent, convolution, and attention-KV state must all describe the same materialized prefix. A newly sampled correction or bonus token can remain pending until the next forward.

visibility, physical packing, path scheduling, and the acceptance walk from the same logical topology. Padding permits reusable execution shapes while remaining invisible to attention and selection. The particular tree geometry used in our experiments is specified in Section VI.

The attention layout places the principal spine first in the physical suffix and permutes KV write locations and ancestry-mask key columns while retaining logical query-row and position order. Selection remains in logical-node space; accepted-path publication maps those nodes back to their physical slots. This distinction prevents a layout optimization from silently changing which key/value entries a branch reads. Recurrent paths begin at the native pre-tree state or a transient cut state; the device committer publishes the selected recurrent and convolution state into native running rows and re-linearizes its attention KV entries [6], [7].

Algorithm 1 separates candidate verification, path selection, and state publication. Every publication operation uses the same materialized path A . A newly sampled token b^* can be emitted while remaining pending: its state is created by the next target forward.

IV. GPU SCAN AND ACCEPTED-PATH REPLAY

A. Path-parallel verification and state lifetime

The GDN verifier decomposes the tree into dependent path groups, a scheduling family also studied by SpecLA [8]. A first path starts from the request’s pre-tree state and exports the cut states needed by downstream paths. The second level evaluates independent branch paths from those cut states. Each GPU program covers one path, head, and value tile, retaining the full key dimension in its working state tile. Sequential updates reuse this tile through the path loop rather than loading a full recurrent state for every node. Candidate outputs continue through the model; rounded operands are recorded for later accepted-path replay [6]. The source expresses this local reuse; register allocation and spills remain compiler- and geometry-dependent.

Cut states and operand rings are temporary within a verification step. The durable continuation resides in the request-owned native recurrent and convolution running rows. Replay starts from the pre-tree running state and publishes the accepted frontier back to those rows; prefix-cache snapshots and restores use that native layout. Static work buffers can be reused by

CUDA graphs, but they do not replace the request’s persistent state with a branch-leaf pointer.

The two levels execute in separate GPU launches and exchange only the boundary states required by downstream paths. This schedule exposes parallel branch work while retaining the state-transfer cost at the decomposition boundary.

B. Sequential arithmetic and accepted-path replay

Verification uses sequential path scans with fp32 carried state, raw gate inputs, and in-kernel query/key normalization. Accepted-path replay uses native fused GDN updates with recorded accepted operands and in-place publication to the fp32 state bank. Its graph contains one native update call per GDN layer. Query/key normalization, beta conversion, gate evaluation, and output casts can differ between these implementations; agreement is not guaranteed by their common real-arithmetic recurrence [6].

The sampling stage produces accepted node indices and path lengths on the GPU. Recurrent replay, convolution publication, and attention-KV publication consume this same path. Graph capture reduces host launch overhead for the sequence of per-layer native updates; each layer retains its own update kernel.

C. Temporary state versus recomputation

Path decomposition caches the states needed where downstream paths branch, then reuses them across independent work. If a program exports N_c cut states with value-tile width B_v and key dimension d_k , their fp32 payload is

$$M_{\text{cut}} = 4N_c B_v d_k \text{ bytes.} \quad (4)$$

This counts one head and value tile’s logical export payload. The current reusable scratch allocation reserves a full matrix slot for each actual node even though only required cut states are written; allocated bytes therefore differ from state traffic. Recomputing ancestry could remove some exports at the cost of repeating updates. Exporting every candidate state would instead trade more writes for less recomputation. The two-level path schedule occupies a particular point in this design space.

Accepted-path replay carries one state tile through a bounded path loop. Its carried-state footprint is $O(B_v d_k)$ in tree size, while the recorded operands and candidate outputs scale with the verification shape. Static padding makes graph buffers and kernel geometry reusable, but padded rows must remain invisible to attention and selection. Neither a shorter logical path nor fewer state writes alone establishes a faster complete request.

The closest scheduling comparison is SpecLA: both systems carry state tiles through sequential paths and parallelize ready paths. Their continuation policies differ. SpecLA buffers accepted factors for application in a later fused update [8]; LumoTree immediately replays recorded operands through native per-layer updates and publishes the matching convolution and attention state. Weaver also uses selected-path replay, but verifies candidates with a triangular solve [9]. Bole reconstructs accepted states from compact factors in a batched commit [30]. TreeWY can fuse prior-frontier reconstruction

Algorithm 1 Tree verification and native continuation

Input: Native continuation state; candidate tree T ; proposal probabilities q

Output: Emitted tokens; updated native state; next drafter input; any pending token

Verify candidates

- 1: Map T to physical slots, positions, and ancestor masks.
- 2: $p \leftarrow$ target probabilities from path-local GDN, convolution, and attention computation.
- 3: Retain candidate hidden states, rounded replay operands, convolution histories, and temporary KV entries.

Select one continuation

- 4: $(A, b^*) \leftarrow$ accepted materialized path and pending token from sampling with p and q .

Publish the same path across all state components

- 5: **for** each GDN layer **do**
 - 6: Replay recorded operands along A into the native recurrent state.
 - 7: Gather the convolution history for A into the native running row.
 - 8: **end for**
 - 9: Remap the accepted KV entries using the same logical-to-physical slot map.
 - 10: Select the matching hidden-state row for the next drafter input.
 - 11: Advance positions along A ; keep b^* pending until its target forward.
 - 12: **return** Emitted tokens and the updated continuation boundary.
-

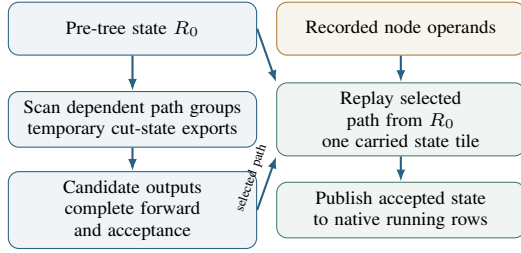


Fig. 3. State lifetime in the path-scan verifier. Temporary cut states connect path groups; recorded operands support selected-path replay. Only the accepted frontier becomes native persistent state.

with the next verification while retaining its working tile [32]. Thus, LumoTree pairs path recurrence with immediate native replay and coordinated hybrid-state publication. This pays explicit accepted-path update work in exchange for returning each iteration to the native state representation. Complete-cycle measurements must include that tradeoff.

V. OPTIMIZING THE LUMOTREE VERIFICATION CYCLE

The architectural choice in Section IV is realized through the implementation changes below. Three representations must agree: logical tree nodes, temporary verification storage, and the native state used by the next forward. LumoTree carries one accepted path across these representations and reorganizes preparation and selection around it, using established techniques such as fusion and graph replay. The nontrivial constraints arise at their boundaries: changing a slot order, a rounding step, or a tied candidate can change which continuation the next iteration consumes. Table II summarizes the changes and these constraints.

A. One device-selected path drives native continuation

Selecting valid output tokens does not by itself update a hybrid model’s state. The recurrent matrix must contain the accepted updates, convolution must retain the accepted history, attention must reference the accepted KV entries, and the drafter must receive the corresponding hidden row. Reconstructing these choices separately on the host introduces path-list preparation and synchronization between acceptance and publication.

LumoTree replaces that handoff with fixed-shape device products: accepted node indices, path lengths, emitted tokens, and the selected final row. These products feed native recurrent replay, convolution gathering, KV remapping, and drafter-row selection. Recorded operands are staged for the native per-layer updates in one captured committer graph; convolution and KV publication use the same selected path [6]. Graph capture amortizes launch orchestration, while the shared path prevents each state component from independently interpreting acceptance.

The distinction between emitted and materialized tokens must survive this handoff. A correction or bonus token may be emitted while remaining pending for the next target forward; padded path entries must not advance state. After publication, the implementation stores persistent state in native request rows and branch cut states in temporary scratch. Retaining that native representation supports the intended prefix-cache and serving-graph interface; it does not by itself qualify cold/hit behavior, row reuse, or stale-state refusal. The implementation captures the serving, drafter, and committer stages separately.

B. One slot map governs attention and publication

An ancestry mask specifies visible keys, but does not specify their physical placement or the reduction groups that process them. Interleaving masked branch columns with spine keys can

Algorithm 2 Path-parallel scan and accepted-path replay

Input: Candidate tree T and the pre-tree recurrent state for each GDN layer

Output: Recurrent outputs for all candidates; native recurrent state for the accepted path

Verify paths; keep branch state temporary

```
1: for each GDN layer reached during the target forward do
2:   Compute and record this layer's rounded node operands.
3:   for each path group, in dependency order do
4:     parallel for each ready path, head, and value tile do
5:       Load the working state tile from the pre-tree state or a saved parent cut state.
6:       for each node on this path, in ancestry order do
7:         Update the working tile with this node's operands; write its recurrent output.
8:         Save the tile if a downstream path needs this node's state.
9:       end for
10:    end for
11:  end for
12: end for
```

Finish verification and select a path

13: Complete the target forward and sampling to obtain the accepted materialized path A .

Replay only the accepted path into native recurrent state

```
14: for each GDN layer do
15:   Start from this layer's pre-tree recurrent state.
16:   for each node on  $A$ , in path order do
17:     Apply the native GDN update using this layer's recorded operands.
18:   end for
19:   Publish the resulting recurrent state to the request's native running row.
20: end for
```

TABLE II

CONCRETE IMPLEMENTATION CHANGES AROUND THE PATH VERIFIER. THE THIRD COLUMN IDENTIFIES THE COORDINATION OR NUMERICAL CONSTRAINT THAT MAKES EACH CHANGE NONTRIVIAL. WORKLOAD RATES MEASURE THEIR COMBINED EXECUTION.

Starting work or constraint	LumoTree change	What must remain consistent
Host path-list construction between acceptance and publication	Device-selected path feeds captured native replay, convolution, KV and drafter selection	All consumers select the same materialized prefix; pending and padded tokens do not advance state
Logical ancestry does not determine physical KV placement	Spine-contiguous slots with coordinated writes, mask columns and accepted-path remap	Logical query order, physical source slots and native destination positions agree
Per-node convolution histories and repeated layer preparation	Static ancestry gathers, fused ordered taps and per-layer operand buffers	Tap order, cast boundaries, history padding and layer identity
Separate score preparation and argmax/top-k operations	One shared logits tensor and fused selection into graph buffers	Both reference tie conventions, candidate order and duplicate behavior
Repeated KV staging and limited decode parallelism	Grouped query heads and split-K in the patched tree-attention kernel	Tree visibility and slot mapping across partitions; characterized reduction error

change the online-softmax reduction layout even when logical visibility is unchanged. Physical ordering therefore matters both for addressing and for finite-precision execution [35].

LumoTree places the principal spine contiguously in the candidate suffix while retaining logical query-row and position order. It applies the same permutation to KV write slots and ancestry-mask key columns. Acceptance remains in logical-node space; publication translates the selected nodes through that slot map into native continuation positions [6]. Reordering only writes, only masks, or only the final copy would leave another consumer reading a different branch. The implementation change is this coordinated mapping across verification

and publication.

The patched FlashAttention-2 decode kernel combines this layout with grouped query heads that share staged KV tiles and split-K context partitions. KV reuse and partitioned attention are established techniques [23], [24]; here they are applied under the tree's visibility and slot mapping. They target repeated KV staging and insufficient parallel work at batch one. Prefill retains a native-compatible attention path. The current component tests measure repeatability and numerical error of the deployed attention kernel; the isolated effect of spine placement remains an ablation question.

C. Precompute branch histories while preserving arithmetic

The starting tree-convolution implementation constructs candidate histories with per-node operations and repeats path-row preparation across layers. LumoTree derives static gather indices from the topology, gathers windows from the prior convolution history and candidate inputs, and fuses the tap computation. Stacked operand buffers retain each GDN layer’s inputs for subsequent accepted-path replay [6].

This transformation must preserve more than ancestry. Convolution tap order and cast boundaries affect the values recorded for recurrent updates, and unused history positions must retain the intended padding. A generic reduction can compute the same real-arithmetic sum in a different floating-point order. The fused implementation therefore retains the ordered tap operations and conversion points while replacing repeated history construction with prepared data movement. Each layer still records and replays its own operands.

D. Fuse draft selection without changing proposals

At each MTP head depth, LumoTree computes one full-vocabulary logits tensor and reuses it for the principal token and sibling candidates. A fused CUDA kernel writes the spine token and three ordered candidate indices directly into reusable graph buffers, replacing separate argmax/top-k reductions and their output copies. The principal chain advances through captured MTP iterations; sibling selection reuses its scores, and suffix prediction extends selected paths within the fixed verification capacity [6].

The subtlety is that argmax and top-k do not have interchangeable tie behavior. The reference argmax chooses the lowest maximal token index, while its top-k operation orders selected ties differently. Simply taking the first top-k result as the spine token can change the proposal tree. Our fused selection preserves both reference outputs, including duplicate proposals where produced by the reference. Exact-output and captured-replay tests check this component. The optimization removes redundant score preparation and selection work while retaining the vocabulary projection.

VI. EVALUATION SETUP

We evaluate Qwen3.8-27B NVFP4 on a single GB10/DGX Spark using exploratory coding-agent deployments on SWE-bench Verified Astropy tasks [6], [10]. Three LumoTree builds and two SGLang EAGLE runs share a two-task subset; a separate LumoTree deployment covers ten tasks. The builds differ in implementation revision and are not statistical replicates. We report complete recorded outcomes and distinguish the shared-task comparison from the larger deployment. All decode rates pool token counts and decode durations over the same completed-request population, excluding time to first output. Agent duration additionally includes tools and repository operations; server startup and subsequent evaluation are separate.

The evaluated tree has 27 draft candidates and one root, padded to 32 verification rows. A five-depth MTP head supplies 15 candidates; suffix prediction adds a six-token principal tail

and rescue branches of four and two tokens. Maximum draft depth is eleven, and four post-root MTP forwards construct the head. Attention groups query heads in pairs and divides the context into four partitions. The model has 48 GDN layers and sixteen full-attention layers. All LumoTree workload deployments use this geometry and the patched FlashAttention-2 path. Appendix A gives the serving configuration; the accompanying artifact identifies each source and kernel version.

VII. RESULTS

A. SWE-bench Verified coding-agent workload

The agent uses Qwen Code to inspect repositories, edit files, execute tools, and produce patches for Astropy tasks from SWE-bench Verified. Model serving runs on GB10; task execution and evaluation use x86-64 workers. We distinguish a shared two-task comparison from a separate ten-task deployment. Appendix B reports the complete recorded outcomes, including unsuccessful attempts and runs excluded from the shared comparison.

a) *Task outcomes*: The ten-task deployment resolves six tasks; four patches fail tests. All ten attempts use tools and produce nonempty patches. Agent durations range from 1.67 to 41.24 minutes and sum to 181.15 minutes, excluding server startup and subsequent evaluation. These selected tasks have one attempt each and no matched native-decoding control, so their resolution count is not a benchmark-wide success estimate or evidence of quality preservation.

The shared comparison uses Astropy tasks 12907 and 13033. Each of the three LumoTree builds and the included SGLang EAGLE run resolves the first task and fails the second. Their different token trajectories and agent durations do not isolate verification cost. For example, Build A accumulates 17.31 agent minutes across the pair, compared with 34.07 for the included SGLang run. One recorded pair per build cannot establish a general task-completion speedup.

b) *Decode throughput*: For completed request r , let n_r be the output-token count, ℓ_r the server-recorded end-to-end latency, and f_r the time to first output. We report

$$D_{\text{pool}} = \frac{\sum_{r=1}^R (n_r - 1)}{\sum_{r=1}^R (\ell_r - f_r)}. \quad (5)$$

Counts and durations are pooled over the same completed requests before division. The numerator counts output intervals; the denominator excludes first-output latency and between-request agent/tool work. Overlapping requests contribute their durations separately, so this estimator is not aggregate throughput per elapsed wall-clock second or a measure of isolated GPU utilization. All completed engine requests, including internal compaction traffic, are included.

c) *Selection and comparison limits*: We selected the shared tasks retrospectively because both had completed SGLang records, then included every recorded LumoTree run on that pair. The performance subset requires both agent attempts to finish normally without a timeout or terminal budget cap and produce nonempty patches; patches that fail

TABLE III

POOLED DECODE RATES ON GB10. BUILDS A–C AND SGLANG USE THE SAME TWO ASTROPY TASKS, 12907 AND 13033, AND PRODUCE NONEMPTY PATCHES ON BOTH. THE FINAL ROW COVERS A SEPARATE TEN-TASK SUBSET. BUILDS A–C ARE DEVELOPMENT REVISIONS, NOT STATISTICAL REPLICATES.

Deployment	Tasks	Requests	Tokens/s
SGLang EAGLE	2	45	26.89
LumoTree Build A	2	41	29.09
LumoTree Build B	2	35	28.28
LumoTree Build C	2	53	27.27
LumoTree ten-task cohort	10	265	25.63

tests remain included. Three LumoTree pairs and one of two SGLang pairs satisfy this rule. The excluded SGLang pair has an unfiltered rate of 30.70 tokens/s, but its second task produces an empty patch after a thinking-only response reaches the output cap. Appendix B retains this failure and its rate. The selected rates therefore describe patch-producing runs, rather than performance over all attempts.

The systems share the requested sampling settings and agent harness, but differ in checkpoint conversion, attention implementation, scheduling, context limits, and proposal budgets (Appendix A). Cross-stack weight and tokenizer identity is not verified. Builds A–C also differ in implementation revision, so their spread does not estimate within-build uncertainty. The ten-task cohort is a completed segment of an interrupted development campaign; other segments include degeneration, incomplete output, and budget-capped attempts. These observations demonstrate execution of real coding-agent work, but do not establish a same-stack speed advantage or general task-quality preservation.

B. Acceptance and execution costs

Output-side acceptance and GPU phase measurements describe the recorded workloads. Accepted drafts per speculative event range from 4.41 to 4.61 for the shared-task builds and average 4.05 in the ten-task deployment. These counters measure emitted draft length after filtering, excluding the correction or bonus token. They do not identify the accepted branch or the path materialized in persistent state. Physical draft slots include padding, so their count is not an appropriate denominator for logical-candidate acceptance.

Each timer is reduced using its own duration sum and span count, with no pending samples at task boundaries. Dispatch does not separate sampling from native replay. Instrumentation overhead was not measured through an on/off control, and the recorded spans do not supply isolated attention, convolution, or replay costs. SGLang’s saved acceptance statistics are window gauges rather than task-total counters, preventing a directly comparable acceptance estimate.

C. Numerical and candidate-selection validation

We test GDN verification and accepted recurrent-state publication on two calibration and two held-out synthetic operand sets. Each set supplies 48 recurrent instances with the

TABLE IV

ACCEPTED DRAFTS PER EVENT AND MEAN GPU SPAN DURATIONS (MS). DISPATCH COVERS ACCEPTANCE, PATH HANDLING, AND STATE COMMITMENT. THESE OBSERVATIONAL MEASUREMENTS DO NOT ISOLATE OPTIMIZATION EFFECTS OR SUM TO A VERIFIED CRITICAL-PATH DECOMPOSITION.

Deployment	Accepted			
	drafts	Target	Drafter	Dispatch
Build A	4.61	113.89	51.95	20.73
Build B	4.49	114.57	52.49	20.68
Build C	4.41	115.90	54.61	20.46
Ten-task cohort	4.05	114.89	54.76	20.15

TABLE V

GDN COMPONENT CHECKS ON GB10. BITWISE COLUMNS COMPARE WITH THE NATIVE CHAIN; THE PAIRED RULE COMPARES BOTH IMPLEMENTATIONS WITH A FLOAT64 REFERENCE. COUNTS ARE PER BLOCK AND PROCESS.

Operand sets	BF16 output	FP32 state	Paired rule passed
	bitwise	bitwise	
Calibration	3,004/3,072	2,688/2,688	5,760/5,760
Held-out	3,018/3,072	2,688/2,688	5,760/5,760

evaluated tree geometry: 32 physical output rows, including padding, and 28 accepted publication paths. Each block runs twice in each of two fresh processes. These repetitions check reproducibility; they do not increase the number of distinct inputs.

For each head, we compare both implementations with a float64 recurrence. The rule, frozen before candidate evaluation, requires both root-mean-square and maximum absolute errors to satisfy

$$E_{\text{tree}} \leq 1.10E_{\text{native}} + 2^{-24}M_{64} + 2^{-149}, \quad (6)$$

where M_{64} is the corresponding root-mean-square or maximum magnitude of the reference. Table V reports all enumerated cases passing in both processes. All published recurrent states are bitwise equal to the native chain, while 68 calibration and 54 held-out output tensors differ in BF16 rounding. Native and candidate results reproduce bitwise across repeats and processes.

Calibration controls detect sibling substitution, swapped state rings, and stale publication metadata. Poisoning an off-path branch leaves the tested selected-path state unchanged, and a zero-replay control is rejected. These results support the tested GDN scan and recurrent publication mechanism. Complete convolution/KV continuation and next-forward model decisions require the separate full-model qualification.

a) *Full-model native reference*: Before comparing the tree route with native continuation, we test whether the native reference reproduces the state and decisions that define that comparison. Each condition covers 84 predetermined paths: a root-only continuation and 27 candidate paths for each of three calibration prefixes. Every path is repeated twice in each of two independent processes. The conditions distinguish natural prefill from an imported common starting state and separate

TABLE VI

CROSS-PROCESS NATIVE-REFERENCE AGREEMENT, IN PATHS OUT OF 84. STATE/LOGITS REQUIRES IDENTICAL PUBLISHED TARGET STATE AND FULL TARGET LOGITS. JOINT CHOICES REQUIRES THE NEXT TARGET DECISION AND THE ACTUAL MTP SPINE AND ORDERED TOP-THREE SELECTIONS AT EVERY TESTED PHASE. A DASH DENOTES AN UNTESTED JOINT MTP CONDITION.

	Greedy	State/ logits	Joint choices
Starting-state control			
Natural prefill	70/84	0/84	–
Common target state	84/84	84/84	–
Common target and MTP state	84/84	65/84	68/84
With fixed kernel selection	84/84	65/84	68/84

target continuation from continuation coupled to the MTP drafter.

Natural-prefill continuation reproduces within each process but changes the greedy winner on fourteen paths across processes. The target-only control with common starting state reproduces identical target state, logits, and greedy decisions on all paths. When MTP continuation is included, greedy decisions remain stable on all paths, but target state and full logits agree on only 65, and joint target/draft choices on 68. Holding kernel selection fixed yields the same aggregate counts.

All sixteen paths with differing joint choices have long prefixes. Their within-process repetitions agree, and every principal MTP spine remains stable; the disagreements affect secondary top-three membership or order. The tested phases include initial MTP continuation, a follow-up, and continuation after consuming the pending target token. Common imported state and matched tokens and positions do not guarantee identical intermediate hidden tensors. These observations therefore do not isolate nondeterminism in an MTP kernel given identical hidden/cache operands, nor do matching aggregate counts establish a cause.

Stable greedy output alone is insufficient to establish reproducible proposal trees or full-model state equivalence. The joint native reference does not satisfy the specified agreement condition, so a full-model tree-candidate comparison was not executed under this protocol. This is a reference limitation, not a measured candidate failure. Multi-cycle continuation, prefix-cache lifecycle behavior, and distribution preservation remain unestablished.

The fused full-vocabulary selection test compares the two output tensors against the reference argmax and top-k operations. Across 1,368 input cases and five block-count settings (6,840 configurations), it records zero byte mismatches; planted negative controls fire. A separate captured four-level selection graph completes 24 replays with zero mismatches. These results validate candidate-selection outputs for the tested inputs and graph behavior, not end-to-end task equivalence [6].

Attention tests use the same kernel build as the ten-task deployment. They show bit-identical repeated output and log-sum-exp values within each of sixteen determinism cases, with matching digests across two processes. On synthetic query/key/value tensors drawn at scales measured from banked

TABLE VII

UNTIMED RECURRENT-COMPONENT COMPARISON ON GB10. FAILED CELLS PER REPEAT COMPRISE 64,512 OUTPUT CELLS AND 6,912 STATE CELLS PER CONFIGURATION. BOTH REPEATS GIVE THE SAME COUNTS. EACH IMPLEMENTATION IS TESTED AGAINST ITS SPECIFIED COMPARATOR, AS DESCRIBED BELOW.

Configuration	Output failures	State failures
LumoTree / native	0	0
Weaver author-default	53,745	1,154
Weaver aligned-local	57,707	388
TreeWY author-default	37,528	6,912

model operands, 93.31% of output elements agree with the reference kernel within two ULP, the maximum absolute output difference is 0.00390625, and maximum log-sum-exp disagreement is four ULP. All nine declared checks pass, including comparisons against a float64 dense reference and checks for non-finite disagreements. These are attention-component bounds; split-K changes rounding and is not claimed to be byte-identical to the reference kernel or to preserve every downstream token.

Runtime traces confirm execution of the specialized kernel in all sixteen attention layers, fused selection, device acceptance, and captured state publication during the workload runs. Kernel execution and observed task outcomes do not establish full-model numerical equivalence or distribution preservation.

D. Recurrent verification and publication comparison

We compare recurrent verification and accepted-state publication using held-out synthetic operands spanning 48 recurrent instances and 48 value heads. Verification covers the tree’s 28 active nodes. Three accepted-path publications produce a history of 1, 6, and 11 updates, including Weaver’s accepted replay and TreeWY’s final deferred commit. Each configuration is evaluated twice in one process, with a fresh initial state for each cycle. A numerical cell fixes the recurrent instance, output node or publication, and value head; errors are reduced over the remaining vector or matrix entries. There are 71,424 distinct cells per configuration.

Each cell must satisfy both root-mean-square and maximum absolute error bounds,

$$E_{\text{method}} \leq 1.10E_{\text{seq}} + 2^{-24}M_{64} + 2^{-149}, \quad (7)$$

where errors are measured against the specified float64 recurrence and M_{64} is its corresponding root-mean-square or maximum magnitude. The bounds are fixed before evaluating candidate outputs. For LumoTree, the sequential comparator is the native GPU recurrence. The other implementations use predeclared sequential software comparators that do not emulate every tensor-core rounding operation.

TreeWY retains the author implementation’s normalization floor of 10^{-12} and BF16 dot products. Weaver uses TF32 verification, FP32 elementwise accepted replay, and an additive normalization constant of 10^{-6} . We report both its author-default preparation and a separately labeled local FP32-

preparation variant. These preprocessing and arithmetic choices remain distinct rather than being silently aligned.

LumoTree satisfies every output and publication bound. The three comparator configurations fail their specified bounds, while all configurations produce finite tensors and reproduce bitwise between repeats. TreeWY’s BF16 compact commit and the sequential comparator’s FP32 outer-product updates, for example, are different floating-point computations. These results characterize agreement with the specified references; they do not establish algorithmic incorrectness or a task-quality ranking. We consequently report no timing ranking from these configurations. The experiment covers recurrent computation, not full-model convolution, attention-cache, or drafter continuation.

VIII. DISCUSSION

The design couples tree geometry, physical layout, and continuation publication. Optimizing any one of these in isolation can move work elsewhere or invalidate another stage’s mapping. The verifier uses transient cut states, static graph buffers, and native persistent rows for different purposes; conflating them would obscure both memory cost and state ownership.

Sequential replay and compact reconstruction remain alternative implementation choices. The held-out recurrent comparison includes accepted publication and final deferred commitment under each implementation’s stated preprocessing and arithmetic conventions. Only LumoTree satisfies its specified numerical envelope. The differing comparators and failed bounds limit this result to numerical behavior; it supplies no timing ranking or complete-system advantage.

For long-running agents, task completion and task wall time are the relevant application outcomes. More decoded tokens can reflect additional reasoning or a longer tool loop. The pooled decode comparison is therefore accompanied by task identities, outcomes, agent/tool policy, and terminal budgets. It describes the evaluated deployments and the stated retrospective patch-producing subset; it is not a task-completion speedup estimate.

Component validation and whole-task performance answer different questions. The selection test checks exact candidate outputs; the split-K test checks deterministic bounded rounding; the workload tests exercise the composed system. A broader quality claim requires held-out tasks and repetitions. Isolating an optimization’s application benefit requires toggling that component within the same eligible serving configuration and task protocol.

IX. CONCLUSION

LumoTree executes recurrent paths in parallel, reuses each working state tile across path updates, and publishes one accepted continuation across the hybrid model. A shared descriptor coordinates attention visibility, branch-local GDN computation, convolution history, device probability arithmetic, and native continuation. Sequential scan and accepted-path replay make the state lifetime and arithmetic choices explicit;

cached state, recomputation, physical layout, and independent-work batching expose the main optimization tradeoffs.

Exploratory Qwen3.8-27B NVFP4 deployments on GB10 give 27.27–29.09 pooled tokens/s across three tree revisions and 26.89 for SGLang EAGLE on two shared SWE-bench Verified Astropy tasks with nonempty patches. The separate ten-task deployment attains 25.63 tokens/s, with six tasks resolved and four failing tests. The all-attempt ledger also reports the excluded empty-patch SGLang pair and additional incomplete records. These observations describe the recorded systems and populations; fixed-build repetitions and same-stack controls are required for a confirmatory advantage claim.

The contribution is a concrete verifier design and component-level numerical evidence, accompanied by exploratory application results and explicit limitations. Native-reference differences leave full-model continuation and cache lifecycle correctness unestablished. The recurrent-component comparison supplies numerical results rather than a timing ranking. Same-stack task comparisons and controlled ablations are needed to establish application speedups and attribute them to individual design choices.

APPENDIX A

SERVING CONFIGURATION AND MEASUREMENT

The artifact provides source versions, kernel identities, launch settings, task metadata, and raw measurements. Earlier implementations are documented separately and are not pooled with the evaluated NVFP4 deployments. The LumoTree engine is a patched vLLM development build. It serves a RadixArk Qwen3.8-27B NVFP4 checkpoint with ModelOpt mixed quantization and bfloat16 execution. SGLang serves its recorded as-shipped NVFP4 model. The artifact identifies the evaluated builds and available weight/tokenizer metadata; identical model names do not establish identical checkpoint tensors or conversion policies.

The ten-task deployment uses engine seed zero, a 20,000-token compaction cap, and a 9,000-second agent timeout. Tasks run in repository images on an internal network that permits the model proxy and blocks external egress. The included SGLang pair uses the same agent harness and requested sampling settings. Build A’s 41 responses and the included SGLang run’s 45 responses are all at most 20,000 tokens, below both output caps. This observation does not resolve the other differences between the serving configurations.

Workload rates use Eq. 5; token counts and latency sums cover the same completed requests. Metric counters are checked for continuity, resets, and outstanding requests at task boundaries. First-output latency, tools, and evaluator time remain distinct from decode duration. GPU phase times divide each timer’s duration sum by its own span count. Output-side acceptance counters describe emitted draft lengths and do not identify the committed tree path.

Algebraic correctness concerns the real-arithmetic recurrence. *Numerical agreement* concerns particular kernels and references. *Sampler correctness* concerns the acceptance/residual law for the supplied probabilities. *Serving correctness* additionally

TABLE VIII

SERVING CONFIGURATIONS FOR THE EXPLORATORY WORKLOAD EXPERIMENTS. BUILDS A–C SHARE THE TREE GEOMETRY BUT DIFFER IN IMPLEMENTATION REVISION. THE CONFIGURATIONS ARE NOT A CONTROLLED SAME-STACK COMPARISON. EXACT SOFTWARE AND KERNEL VERSIONS ARE PROVIDED WITH THE ARTIFACT.

Setting	LumoTree / patched vLLM	SGLang EAGLE
Target	Qwen3.8-27B NVFP4, RadixArk conversion; bf16 execution	Qwen3.8-27B NVFP4, recorded as-shipped checkpoint
Weight/tokenizer identity	Model/configuration metadata recorded; cross-stack byte identity unverified	Loaded weight/tokenizer hashes incomplete
Proposal configuration	27 drafts; root plus drafts padded to 32 rows; maximum depth 11	Steps/top- k /draft tokens: 3/1/4
Attention	Patched FlashAttention-2; head grouping 2, split-K 4	FlashInfer
Graphs/cache	Prefix caching; full/piecewise target graphs; separate drafter and committer graphs	Exact execution-mode parity not established
Context/scheduling	131,072 context; 4,096 scheduled tokens; one active sequence	262,144 context; 8,192-token prefill chunks; up to three observed requests in the excluded run
Response cap	32,768 for Builds A–C; 24,000 for ten-task cohort	32,768 for excluded run; 24,000 for included run
Requested sampling	Temperature 0.6; top- p 0.95; top- k 20; min- p 0; presence penalty 1.0	
Agent protocol	Qwen Code 0.19.4; one task at a time; proxy auto-continuation disabled	

requires coherent state across continuation and cache operations [2], [6], [26]. Component outputs and task completion address different parts of this chain.

For target probabilities p and proposal probabilities q , ordinary rejection sampling combines accepted mass $\min(p, q)$ with residual mass proportional to $(p - q)_+$. This gives the intended target marginal only for the probabilities actually supplied [1], [2]. A correct abstract sampling rule cannot compensate for stale state or different target logits. The paper therefore does not promote component checks or task outcomes into a general distribution-preservation claim.

APPENDIX B ADDITIONAL WORKLOAD RESULTS

Table IX lists every task in the ten-task deployment. It covers the remaining tasks of an interrupted sixteen-task development campaign, rather than a prospectively sampled benchmark cohort. All attempts use tools and produce nonempty patches. No malformed tool-call arguments or degeneration are detected by the recorded trace screen. That screen requires at most 200 visible characters, no tool calls, and at least 2,000 thinking characters; it can miss loops that continue narrating or using tools, so zero detections do not establish general robustness.

The ten-task rate pools 248,342 output tokens over 265 completed requests, with 10,667.62 seconds of summed end-to-end latency and 989.82 seconds to first output. These requests comprise 256 normal and nine compaction requests. The included SGLang pair pools 47,854 tokens over 45 requests, with corresponding latency sums of 2,013.63 and 235.95 seconds. These totals reproduce Table III; raw precision is retained in the artifact.

Table X includes all 22 unique attempts with complete task metric intervals. Build A’s two-task subset is also contained in its four-task row; these rows must not be counted as independent cohorts. The excluded SGLang run remains visible even though its second task produces an empty patch and does not reach test evaluation. Its response cap is larger than that of

TABLE IX

TEN-TASK LUMOTREE DEPLOYMENT. TASK IDS HAVE PREFIX `ASTROPY__ASTROPY-`. AGENT TIME INCLUDES MODEL SERVICE AND TOOLS; EVALUATION OCCURS SUBSEQUENTLY. THERE IS ONE ATTEMPT PER TASK AND NO MATCHED NATIVE ARM.

Task suffix	Agent time (min)	Evaluation
13977	24.59	Failed tests
14096	22.99	Resolved
14182	7.23	Failed tests
14309	1.67	Resolved
14365	6.78	Failed tests
14369	27.70	Resolved
14508	41.24	Resolved
14539	18.57	Resolved
14598	25.79	Failed tests
14995	4.59	Resolved

the included SGLang run, so the failure cannot be attributed to the latter’s smaller cap.

Build A completes four tasks in total, two resolved and two failed. Subsequent development attempts after Builds B and C include a degeneration event and a task without completed evaluation. Other partial records lack complete metric intervals and therefore have no inferred rates. All such records, source versions, failures, and retries remain in the supporting artifact. They are not combined into a single-build benchmark score or substituted for a matched native-decoding control.

TABLE X

COMPLETE RECORDED WORKLOAD GROUPS, INCLUDING UNSUCCESSFUL ATTEMPTS. AGENT MINUTES SUM TASK DURATIONS, NOT ELAPSED EXPERIMENT WALL TIME. RATES USE EQ. 5. THE EXCLUDED SGLANG PAIR (*) CONTAINS AN EMPTY PATCH; THE BUILD A PAIR OVERLAPS ITS FOUR-TASK ROW. DIFFERENT TASK POPULATIONS ARE NOT RANKED AGAINST ONE ANOTHER.

Deployment / subset	Tasks	Resolved	Agent min	Tokens/s
Build A, shared pair	2	1	17.31	29.09
Build B, shared pair	2	1	18.06	28.28
Build C, shared pair	2	1	48.68	27.27
SGLang excluded pair*	2	1	26.70	30.70
SGLang included pair	2	1	34.07	26.89
Ten-task cohort	10	6	181.15	25.63
Build A, all four tasks	4	2	73.67	26.21

TABLE XI

SHARED-TASK OUTCOMES, INCLUDING THE EXCLUDED SGLANG RUN. ASTROPY TASK SUFFIXES IDENTIFY INSTANCES 12907 AND 13033. AGENT SECONDS INCLUDE MODEL SERVICE AND TOOLS, BUT EXCLUDE EVALUATION. DIFFERENT OUTPUT LENGTHS ARE SYSTEM OUTCOMES AND DO NOT ISOLATE KERNEL COST.

Deployment	Task	Agent s	Output tokens	Evaluation
Build A	12907	407.19	7,873	Resolved
Build A	13033	631.57	17,154	Failed tests
Build B	12907	291.44	5,603	Resolved
Build B	13033	791.87	20,636	Failed tests
Build C	12907	670.87	14,138	Resolved
Build C	13033	2,249.68	56,286	Failed tests
SGLang excluded*	12907	238.72	5,895	Resolved
SGLang excluded*	13033	1,363.45	41,955	Empty patch
SGLang included	12907	426.13	8,314	Resolved
SGLang included	13033	1,617.77	39,540	Failed tests

REFERENCES

- [1] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” 2022. [Online]. Available: <https://arxiv.org/abs/2211.17192>
- [2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, “Accelerating Large Language Model Decoding with Speculative Sampling,” 2023. [Online]. Available: <https://arxiv.org/abs/2302.01318>
- [3] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “Specinfer: Accelerating generative large language model serving with tree-based speculative inference and verification,” 2024. [Online]. Available: <https://arxiv.org/abs/2305.09781>
- [4] S. Yang, J. Kautz, and A. Hatamizadeh, “Gated delta networks: Improving mamba2 with delta rule,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.06464>
- [5] Y. Wu, Z. Qin, A. Wong, and S. Soatto, “Stree: Speculative tree decoding for hybrid state-space models,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.14969>
- [6] Z. Ma, “Lumo FlyWheel implementation and experiment archive,” https://github.com/MaCoredroid/Lumo_FlyWheel, 2026, source versions and experiment records are specified in the accompanying artifact.
- [7] —, “Stateless Tree Speculative Decoding: A Branched Verifier That Caches and Commits Like Native,” https://macoredroid.github.io/Lumo_FlyWheel/stateless-tree.html, 2026, author experiment archive; accessed 2026-09-21.
- [8] Z. Wang, X. Han, Z. Yang, F. Liu, X. Li, R. Gu, S. Zhong, and C. Tian, “SpecLA: Efficient Speculative Decoding for Linear-Attention Models,” 2026. [Online]. Available: <https://arxiv.org/abs/2607.16673v1>
- [9] Y. Oda, R. Mathieu, R. Knyazhitskiy, and A. Chakhvadze, “Trees from Marginals: Autoregressive drafting with factorized priors,” 2026, version 2, July 12, 2026; first submitted July 7, 2026. [Online]. Available: <https://arxiv.org/abs/2607.06763v2>
- [10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” 2023. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [11] Z. Chen, A. May, R. Svirschevski, Y. Huang, M. Ryabinin, Z. Jia, and B. Chen, “Sequoia: Scalable, robust, and hardware-aware speculative decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2402.12374>
- [12] Y. Li, F. Wei, C. Zhang, and H. Zhang, “Eagle-2: Faster inference of language models with dynamic draft trees,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.16858>
- [13] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, X. Duan, Z. Wang, and M. Zhang, “Opt-tree: Speculative decoding with adaptive draft tree structure,” 2025. [Online]. Available: <https://arxiv.org/abs/2406.17276>
- [14] Y. Li, F. Wei, C. Zhang, and H. Zhang, “EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.01840>
- [15] J. Chen, Y. Liang, and Z. Liu, “DFlash: Block Diffusion for Flash Speculative Decoding,” 2026. [Online]. Available: <https://arxiv.org/abs/2602.06036>
- [16] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>
- [17] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient Execution of Structured Language Model Programs,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.07104>
- [18] R. Pan, Z. Wang, Z. Jia, C. Karakus, L. Zancato, T. Dao, Y. Wang, and R. Netravali, “Marconi: Prefix Caching for the Era of Hybrid LLMs,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.19379>
- [19] SGLang Team, “Hybrid Models Meet SGLang: More than Full Attention,” <https://pytorch.org/blog/hybrid-models-meet-sglang-more-than-full-attention/>, 2025.
- [20] SGLang Project, “Qwen3-Next SGLang documentation,” <https://lmsysorg.mintlify.app/cookbook/autoregressive/Qwen/Qwen3-Next>, 2026.
- [21] T. Dao, D. Haziza, F. Massa, and G. Sizov, “Flash-Decoding for long-context inference,” <https://pytorch.org/blog/flash-decoding/>, 2023, october 13, 2023.

- [22] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, “FlashInfer: Efficient and Customizable Attention Engine for LLM Inference Serving,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.01005>
- [23] J. Yao, K. Chen, K. Zhang, J. You, B. Yuan, Z. Wang, and T. Lin, “DeFT: Decoding with Flash Tree-attention for Efficient Tree-structured LLM Inference,” 2025. [Online]. Available: <https://arxiv.org/abs/2404.00242>
- [24] Z. Pan, Y. Ding, Y. Guan, Z. Wang, Z. Yu, X. Tang, Y. Wang, and Y. Ding, “FastTree: Optimizing Attention Kernel and Runtime for Tree-Structured LLM Inference,” in *Proceedings of Machine Learning and Systems*, vol. 7, 2025. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2025/hash/96894468eb44631a32d7ebd56f9892c7-Abstract-Conference.html
- [25] T. Dao, “Flashattention-2: Faster attention with better parallelism and work partitioning,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.08691>
- [26] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim, “Parallelizing Linear Transformers with the Delta Rule over Sequence Length,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.06484>
- [27] Y. Wu, Y. Dukler, M. Trager, A. Achille, W. Xia, and S. Soatto, “Snakes and Ladders: Accelerating SSM Inference with Speculative Decoding,” in *Proceedings of the 4th NeurIPS Efficient Natural Language and Speech Processing Workshop*, vol. 262. PMLR, 2024, pp. 292–304. [Online]. Available: <https://proceedings.mlr.press/v262/wu24a.html>
- [28] L. Zhong, S. Xu, H. Wen, T. Xie, Q. Guo, Y. Wang, and M. Li, “Specmamba: Accelerating mamba inference on fpga with speculative decoding,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.19873>
- [29] H. Borobia, E. Seguí-Mas, and G. Tormo-Carbó, “Component-aware self-speculative decoding in hybrid language models,” 2026. [Online]. Available: <https://arxiv.org/abs/2605.01106>
- [30] L. Wang, Y. Su, X. Wu, C. You, Y. Liu, Z. Qiu, J. Zhang, J. Zheng, F. Liu, J. Zhang, C. Tian, and C. Huan, “Bole: Efficient Tree Speculation for Hybrid-Attention Language Models,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.01651>
- [31] S. M. Ghantasala, “TreeWY: Speculative Verification for Gated DeltaNet Hybrids,” 2026. [Online]. Available: <https://arxiv.org/abs/2608.20961>
- [32] —, “TreeWY: Tree Speculative Decoding for Hybrid GDN Models,” <https://github.com/vllm-project/vllm/issues/54080>, 2026, author implementation RFC; accessed September 26, 2026.
- [33] Dao AI Lab and NVIDIA, “ReplaySSM: Cache SSM Inputs Instead of State,” <https://dao-lab.ai/blog/2026/replayssm/>, 2026, author technical report; accessed 2026-09-26.
- [34] X. Yang, C. Peng, Y. Zhao, L. Zeng, A. Hu, J. Yang, S. Wang, J. Lv, X. Liang, C. Yang, J. Liu, and Y. Qiu, “OneLA: Scaling Linear-Attention Decoding to Large Beams in Generative Recommendation,” 2026. [Online]. Available: <https://arxiv.org/abs/2609.12399v1>
- [35] H. He, “Defeating Nondeterminism in LLM Inference,” <https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/>, 2025, thinking Machines Lab, September 10, 2025.
- [36] G. Oliaro, Z. Jia, D. Campos, and A. Qiao, “SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.04975v3>
- [37] Snowflake Arctic Inference, “Suffix Decoding,” <https://arcticinference.readthedocs.io/en/latest/suffix-decoding.html>, 2026, implementation documentation; accessed September 24, 2026.
- [38] Y. Hu, K. Wang, X. Zhang, F. Zhang, C. Li, H. Chen, and J. Zhang, “SAM Decoding: Speculative Decoding via Suffix Automaton,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.10666v3>